

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards (BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness

of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, wellorganized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

Star Topology using Python

```

# Get number of hosts
n = int(input("Enter the number of hosts: "))
# Central switch
switch = "Central Switch"
print("\nSTAR TOPOLOGY")
print("-----")
# Display topology
for i in range(1, n + 1):
    host = f"Host-{i}"
    length = int(input(f"Enter Cat6 cable length for {host} (in meters): "))

    print(f'{host} ---- Cat6 UTP ({length} m) ---- {switch}')
print("\nVerification:")
print(f'Total Hosts Connected: {n}')
print(f'All {n} hosts are connected through the {switch}.')
print("Star topology created successfully.")

```

OUTPUT:

```

===== RESTART: C:/Users/aswin/OneDrive/Documents/1.py =
Enter the number of hosts: 4

STAR TOPOLOGY
-----
Enter Cat6 cable length for Host-1 (in meters): 10
Host-1 ---- Cat6 UTP (10 m) ---- Central Switch
Enter Cat6 cable length for Host-2 (in meters): 15
Host-2 ---- Cat6 UTP (15 m) ---- Central Switch
Enter Cat6 cable length for Host-3 (in meters): 20
Host-3 ---- Cat6 UTP (20 m) ---- Central Switch
Enter Cat6 cable length for Host-4 (in meters): 12
Host-4 ---- Cat6 UTP (12 m) ---- Central Switch

Verification:
Total Hosts Connected: 4
All 4 hosts are connected through the Central Switch.
Star topology created successfully.

```

UTP Cable Length Validation

```

def validate_segment(length_m):
    if length_m <= 100:
        return True
    else:
        return False, "Error: Cable length exceeds the maximum limit of 100 meters."
# Test Case 1
print("Test Case 1:", validate_segment(50))
# Test Case 2
print("Test Case 2:", validate_segment(100))
# Test Case 3
print("Test Case 3:", validate_segment(120))

```

OUTPUT:

```

===== RESTART: C:/Users/aswin/OneDrive/Documents/l.py =====
Test Case 1: True
Test Case 2: True
Test Case 3: (False, 'Error: Cable length exceeds the maximum limit of 100 meters.')
> |

```

CSMA/CD Simulation in Star Topology

```

import random
import time

# Four hosts connected to a central switch
hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]

# Log file
log = open("csma_log.txt", "w")

def write_log(message):
    print(message)
    log.write(message + "\n")

def transmit(host):
    attempt = 0

    while True:
        attempt += 1

        write_log(

```

```

        f"{host}: Transmission attempt {attempt}"
    )

    # Randomly simulate collision
    collision = random.choice([True, False, False])

    if collision:
        write_log(f"{host}: Collision detected!")

        # Binary Exponential Backoff
        backoff = random.randint(1, 2 ** min(attempt, 5))

        write_log(
            f"{host}: Back-off interval = {backoff} seconds"
        )

        time.sleep(backoff)

        write_log(
            f"{host}: Retrying transmission..."
        )

    else:
        write_log(
            f"{host}: Frame transmitted successfully."
        )
        break

write_log("CSMA/CD Simulation - Star Topology")
write_log("-----")

# Simulate transmission for all hosts
for host in hosts:
    transmit(host)

write_log("-----")
write_log("Simulation completed.")

log.close()

```

```
print("\nLog file 'csma_log.txt' created successfully.")
```

OUTPUT:

```
=====
CSMA/CD Simulation - Star Topology
-----
Host-1: Transmission attempt 1
Host-1: Frame transmitted successfully.
Host-2: Transmission attempt 1
Host-2: Collision detected!
Host-2: Back-off interval = 2 seconds
Host-2: Retrying transmission...
Host-2: Transmission attempt 2
Host-2: Collision detected!
Host-2: Back-off interval = 3 seconds
Host-2: Retrying transmission...
Host-2: Transmission attempt 3
Host-2: Frame transmitted successfully.
Host-3: Transmission attempt 1
Host-3: Collision detected!
Host-3: Back-off interval = 2 seconds
Host-3: Retrying transmission...
Host-3: Transmission attempt 2
Host-3: Frame transmitted successfully.
Host-4: Transmission attempt 1
Host-4: Frame transmitted successfully.
-----
Simulation completed.

Log file 'csma_log.txt' created successfully.
```

StarSwitch Simulation

```
class StarSwitch:
```

```
    def __init__(self):
        self.active_ports = []
        self.broadcast_log = []

    # Add a host to the switch
    def add_port(self, host_id):
        if host_id not in self.active_ports:
            self.active_ports.append(host_id)
```

```

        print(host_id, "connected to the switch.")
    else:
        print(host_id, "is already connected.")

# Remove a host from the switch
def remove_port(self, host_id):
    if host_id in self.active_ports:
        self.active_ports.remove(host_id)
        print(host_id, "disconnected from the switch.")
    else:
        print(host_id, "is not connected.")

# Broadcast frame to all hosts except source
def broadcast(self, frame):
    source = frame["source"]
    message = frame["message"]

    destinations = []

    print("\nBroadcasting frame...")
    print("Source :", source)
    print("Message:", message)

    for host in self.active_ports:
        if host != source:
            print("Frame received by:", host)
            destinations.append(host)

    # Store broadcast operation in log
    self.broadcast_log.append({
        "source": source,
        "destinations": destinations,
        "message": message
    })

    print("\nBroadcast Log:")
    print("Source:", source)
    print("Successfully received by:", destinations)

```



```

# -----
# Demonstration
# -----

switch = StarSwitch()

# Connect multiple hosts
switch.add_port("Host1")
switch.add_port("Host2")
switch.add_port("Host3")
switch.add_port("Host4")

print("\nActive Ports:")
print(switch.active_ports)

# Host1 sends a broadcast frame
frame1 = {
    "source": "Host1",
    "message": "Hello everyone!"
}

switch.broadcast(frame1)

# Remove Host3
print("\nRemoving Host3...")
switch.remove_port("Host3")

print("\nActive Ports:")
print(switch.active_ports)

# Host2 sends another broadcast
frame2 = {
    "source": "Host2",
    "message": "Network announcement"
}

switch.broadcast(frame2)

# Display complete log
print("\n===== FINAL BROADCAST LOG =====")

```

```
for log in switch.broadcast_log:
    print("Source:", log["source"])
    print("Message:", log["message"])
    print("Destinations:", log["destinations"])
    print("-----")
```

output:

```
Host1 connected to the switch.
Host2 connected to the switch.
Host3 connected to the switch.
Host4 connected to the switch.

Active Ports:
['Host1', 'Host2', 'Host3', 'Host4']

Broadcasting frame...
Source : Host1
Message: Hello everyone!
Frame received by: Host2
Frame received by: Host3
Frame received by: Host4

Broadcast Log:
Source: Host1
Successfully received by: ['Host2', 'Host3', 'Host4']

Removing Host3...
Host3 disconnected from the switch.

Active Ports:
['Host1', 'Host2', 'Host4']

Broadcasting frame...
Source : Host2
Message: Network announcement
Frame received by: Host1
Frame received by: Host4

Broadcast Log:
Source: Host2
Successfully received by: ['Host1', 'Host4']

===== FINAL BROADCAST LOG =====
Source: Host1
Message: Hello everyone!
Destinations: ['Host2', 'Host3', 'Host4']
-----
Source: Host2
Message: Network announcement
Destinations: ['Host1', 'Host4']
-----
```

> |